SChain: Scalable Concurrency over Flexible Permissioned Blockchain

Xiaodong Qi[1,3], Zhihao Chen[1,4], Haizhen Zhuo[2], Quanqing Xu[2], Chengyu Zhu[1], Zhao Zhang[1,4*], Cheqing Jin[1,4], Aoying Zhou[1,4], Ying Yan[2], Hui Zhang[2]
[1]East China Normal University [2]Ant Group [3]Nanyang Technological University
[4]Engineering Research Center of Blockchain Data Management, Ministry of Education
{xdqi, chenzh, cyzhu}@stu.ecnu.edu.cn, {zhzhang, cqjin, ayzhou}@dase.ecnu.edu.cn
{haizhen.zhz, xuquanqing.xqq, fuying.yy, shengchu.zh}@antgroup.com

***Abstract*—Permissioned blockchains are being widely applied to solve the trust problem in enterprise collaboration. However, most of these systems suffer from low throughput and flexibility lacking issues. In this paper, we present a blockchain system *SChain* with scalable concurrent execution based on a flexible architecture. SChain separates the functionality of a complete "node" into three sub-functions and assigns them to different peers within every organization. Then each organization can scale each sub-function flexibly with no need for negotiation between organizations. Based on this architecture, SChain explores scalable concurrent execution from two levels. First, SChain takes the advantage of multiple peers to execute transactions collectively, while promising they make the same results as one peer does serially. Second, SChain enables concurrent transaction execution across blocks to utilize the resources of peers fully, breaking up the block-by-block process manner, based on a pipelined workflow. The extensive evaluation results demonstrate that SChain significantly outperforms the serial execution and other competing systems-level approaches.**

***Index Terms*—smart contract execution, parallel transaction processing, blockchains**

I. Introduction

Different from public blockchains designed for cryptocurrency [1–3], permissioned (consortium or private) blockchains, such as Hyperledger Fabric [4], Quorum [5] and Diem [6], are being widely applied to support businesses in enterprise collaboration [7, 8]. In general, such enterprise applications have two signature demands: (1) High performance: these applications commonly deploy blockchain systems in place of traditional databases, while requiring the capacity to process tens of thousands of requests per second. (2) Scalability and flexibility: the system throughput necessitates to be scaled as the number of devoted nodes varies like distributed systems [9, 10], without compromising safety requirements. For example, in *Blockchain-as-a-Service* (BaaS) [11] platforms, users pay for the use of nodes. Therefore, they prefer to add/remove nodes dynamically as the size of their business varies.

Recently, many permissioned blockchain systems are proposed to pursue higher performance. A number of works [12–16] leverage various concurrency control protocols to enable concurrent execution of transactions based on a popular *order-execute-validate* paradigm. Nathan et al. [14] further parallelizes the ordering phase and execution phase to mitigate the cost of ordering. Fabric and its variants [4, 17, 18] realize parallelism in the execution phase, following another *execute-order-validate* (EOV) paradigm. In EOV, transactions are executed concurrently on different endorsement peers, and then sent to the ordering service for consensus. Last, the validation peers validate the agreed transactions, aborting any that violates serializability. NeuChain [19] hides the ordering phase with deterministic ordering based on a novel *execute-validate* paradigm. However, all aforementioned approaches suffer from limited scalability and flexibility, i.e., the overall throughput is constrained by the capacity of every individual node, regardless of the number of nodes involved.

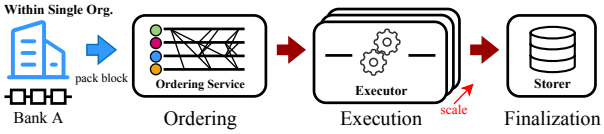


Fig. 1: The order-execute-finalize paradigm.

To enhance system scalability, researchers have incorporated the *sharding* technique into blockchains. The sharded blockchain groups nodes into clusters and let each cluster processes a subset of transactions initiated from its own cluster. However, ensuring the atomicity of *cross-shard transaction* (CST) and the security of every shard presents a significant challenge. Although various sharded blockchains [20–25] have proposed different techniques to address these issues, achieving scalable sharding remains an ongoing challenge. In particular, the addition/removal of shards incurs massive data migration between shards [26]. Additionally, a complicated protocol is required to achieve agreement on shard addition decisions. As more shards are added, the proportion of CSTs increases rapidly, which can significantly impact performance and result in excessive overhead. Hence, this lack of scalability and flexibility is also a fundamental limitation of sharding.

In this paper, we propose a new permissioned blockchain system *SChain*, which achieves scalable concurrent execution based on a decoupling and flexible architecture. First, we observe that the permissioned blockchain is hosted by multiple organizations (e.g., a bank), each of which devotes some peers. Commonly, each organization forms one *trust domain* [4, 27–29], where peers trust each other within the same organization and distrust any peers in other organizations.

*Corresponding author

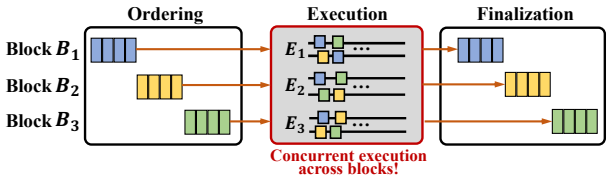


Fig. 2: Scalable concurrent execution based on a pipelined workflow within an organization. There are three peers $E_1 \sim E_3$ to execute transactions, each of which has two threads.

With this observation, SChain separates the functionality of a complete node into three components and assigns them to different peers within every organization, following an *order-execute-finalize* (OEF) paradigm, as shown in Fig. 1. In SChain, peers within the same trust domain (organization) work collectively, so an organization can flexibly add/remove peers on its behalf to adjust its capacity as per the transaction rate, without any communication with other organizations. Besides, the ordering, execution and finalization can be scaled or replaced independently. We focus on how to scale execution in this paper, as there are existing scalable solutions applicable to the other two phases (cf. Section V-D).

Based on the flexible decoupling architecture, we apply the *deterministic concurrency control* to multiple execution peers (executors) in every organization, who execute transactions concurrently and collectively. To enable flexible and scalable execution, executors are stateless (i.e., persist no state data), then each organization can add/remove them without data migration. Besides, SChain pipelines workflows for different blocks, achieving an asynchronous workflow, as illustrated in Fig. 2. Different from existing pipelining approaches, SChain allows transactions in multiple blocks to be executed concurrently, instead of block-by-block execution. Then, every executor can keep executing transactions and avoid staying idle between the time gap of two blocks. SChain ensures the merged results generated by executors take the same effect as that of serial execution on a single executor, which guarantees all organizations enter the same state after execution.

In summary, we have made the following contributions.

- **Flexible architecture.** We propose a novel permissioned blockchain SChain with a flexible architecture. SChain decouples the functionality of blockchain and makes each component scalable independently within every organization.
- **Scalable concurrent execution.** SChain extends concurrent execution to multiple peers in each organization and achieves continuous parallel execution across blocks based on a pipelined workflow.
- **Prototype implementation.** We implement a prototype permissioned blockchain integrating above techniques and conduct experimental evaluations. Extensive experimental results show that our proposed approach outperforms existing competing solutions.

II. Background and Related Work

We provide background information on the blockchain along with related scaling solutions.

A. Blockchain and smart contracts

Blockchain. A *blockchain* is a shared and distributed ledger, which consists of a chain of *blocks*, maintained by a decentralized network of nodes. The nodes verify the validity of new transactions and package them as new blocks, following various consensus protocols. Blockchain systems can be classified into two categories: (1) public blockchains; (2) permissioned (or private) blockchains. Commonly, public blockchains, e.g., Bitcoin [3] and Ethereum [2], are open for everyone who is anonymous, thus there is no trust among nodes. Compared to public ones, permissioned blockchains concern the performance more, e.g., the peak throughput of Ant Blockchain [30] is over 100K. Besides, the permissioned blockchain, sometimes, is not decentralized completely, i.e., the nodes coming from the same organization prefer to trust each other. This feature opens an opportunity for collaboration between nodes instead of replicating function of the blockchain on all nodes. However, existing systems do not take this feature into their design.

Smart contracts. A *smart contract* is a self-enforcing computer program, which allows user-defined contractual rules to be programmed and executed automatically on blockchains. Users can trigger the execution of a smart contract by sending a transaction to the blockchain network. Each smart contract has a persistent storage area that stores data referred to as *states*, e.g., balances. After the execution of transactions in a block, the states would be updated to reflect the effect of execution till the current block. Ethereum makes snapshot S_h of all states by a Merkle tree structure [2, 31] at every block height h. The root hash of Merkle tree is included in the block B_h for authentication purpose.

B. Performance scaling

As permissioned blockchains employ more advanced consensus protocols [32, 33] to enable higher throughput, the bottleneck to support large-scale businesses has gradually shifted to transaction execution. Consequently, various efforts try to accelerate the execution from different aspects.

Parallel transaction execution. A direct idea is to allow transactions to be executed concurrently within every node, which has been well-studied in databases [34–38]. These works ensure *serializability*, where the effect of concurrent execution is equivalent to a serial execution in *some* order. However, the order may vary at different times of execution, which can cause nodes running concurrent processes independently to become inconsistent. In blockchain systems, the parallel execution should adhere to an additional requirement of *deterministic serializability*, as defined in Definition 1.

Definition 1 (Deterministic Serializability (DS))**.** *A schedule of transactions, $\langle T_1, \ldots, T_n \rangle$, is **deterministically serializable** if its effect is equivalent to the serialized execution, which conforms to the transactions' commit order, $\langle T_1, \ldots, T_n \rangle$.*

The deterministic serializability bears a close resemblance to the strict serializability (SS) [39]. SS requires serializability, but also imposes a condition on the order of serial schedule:

a transaction that commits before another transaction starts must appear to happen first. In contrast, in the context of the blockchain, the equivalent order of DS is predetermined by the ordering service, thus DS meets the definition of SS.

Fabric [4] supports concurrent execution by introducing optimistic concurrency control under the EOV paradigm. Following this paradigm, transactions are executed in parallel first in the execution phase, and then some of them, which violate the deterministic serializability, are aborted in the validation phase. Some works [17, 18] optimize the commit order of transactions to lower the abort rate and XOX Fabric [40] adds a second deterministic re-execution phase to minimize invalid transactions. However, the validation phase is performed sequentially, limiting the scalability of execution. Even though some parallel validation techniques are feasible, the overall performance is still restricted by the capacity of every single validator. Other works enable concurrent execution under the traditional order-execute-validate (or order-execute) paradigm, such as OXII [12] (ParBlockchain) and *Serializable Snapshot Isolation* (SSI) [14] (BlockchainDB). OXII uses a *dependency graph* to detect possible conflicts between transactions and allows non-conflicting transactions to be executed in parallel. Similarly, these approaches cannot break the limit of a single node, without considering the collaboration execution.

Sharding. Another popular approach to scale blockchains is sharding [24, 41], which groups nodes into several smaller committees. Then incoming transactions can be processed by different committees in parallel. The unavoidable *cross-shard transaction* (CST), which manipulates data on multiple shards, poses great difficulties for system design. Monoxide [20] proposes eventual atomicity to deal with CSTs. ByShard [21] and AHL [24] utilize shard-based two-phase commit (2PC) protocols that necessitate multiple rounds of inter-shard communication to guarantee the atomicity of CSTs. RingBFT [42] reduces the communication between shards to be linear by adhering shards to the ring order.

However, as the number of shards increases, the CST rate increases rapidly and the communication between shards subsequently dominates the system overhead. Moreover, these systems commonly lack flexibility, i.e., adding new shards to improve the performance will lead to state data migration between shards [26].

Decoupling and pipelining. Some systems decompose the functionality of blockchain to pipeline workflows of multiple blocks [4, 19, 43–45]. As an instance, Fabric decouples execution from ordering, assigns them to different peers, and pipelines the execution, ordering and validation for adjacent blocks. Then the execution and ordering for different blocks can be performed simultaneously and every phase can be scaled and replaced independently. However, these approaches still execute transactions in a block-by-block scheme. ServerlessBFT [46] also separates the ordering, execution and storage of blockchain and assigns them to different kinds of nodes, and then parallelizes the execution for multiple blocks. However, ServerlessBFT depends on an authority, the verifier, to detect potential aborts of conflicting transactions.

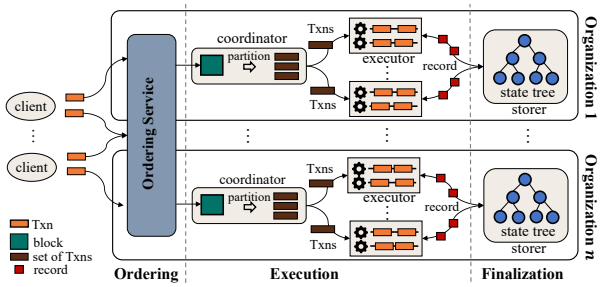


Fig. 3: SChain consisting of n organizations.

III. Overview of SChain

In this section, we overview the architecture of SChain.

A. Architecture

Decoupling. Fig. 3 depicts the overall architecture of SChain that follows the *order-execute-finalize* (OEF) paradigm. Basically, the entire system consists of n *organizations*, each of which plays the role as a complete "node" in conventional blockchain systems. Within an organization, the functionality is decomposed into three services: *ordering*, *execution* and *finalization*. The three services are assigned to different *peers* within every organization. Specifically, the ordering service is in charge of block generation and transactions in every agreed block are delivered to the execution service for execution. The results of the execution are subsequently stored by the finalization service. Under the OEF paradigm, each service can be replaced and upgraded independently, which facilitates pipelined and asynchronous workflows for consecutive blocks.

Trust model. SChain adopts a hybrid trust model. Within the same organization (*trust domain*), peers trust each other naturally but no peers in different organizations. The identities of organizations are publicly accessible and no adversary can create a fake organization to participate in. We deem the hybrid trust model is reasonable for blockchains in a permissioned setting and some works [4, 27, 29] also adopt this model. In conventional blockchain systems [47–49], each node maintains a full copy of blockchain data and takes over the three services: ordering, execution, and finalization (or storage). Sequentially, these three services, located in a same node, trust each other and distrust any services in other nodes. In SChain, peers within the same organization collaborate to maintain the complete functionality of a blockchain "node", and we naturally reserve the trust between services in an organization. An organization can be honest or malicious, but we require that the peers in the ordering service controlled by malicious organizations cannot break the consensus protocol. Specifically, we regard an organization malicious as if one of its peers behaves maliciously.

Workflow & peers. The workflow for every block in SChain experiences threes phases in sequence corresponding to the OEF paradigm. The clients (users) submit their transactions to *orderers* in the ordering service, establishing a total order of transactions grouped by blocks. Within an organization, to

scale the execution, multiple peers called *executors* can be deployed to collectively execute transactions. A designated peer is assigned as the *coordinator* to coordinate the execution over executors. The coordinator partitions the transactions in every block into disjoint sets, each of which is sent to an executor. Then, on every executor, the received transactions are executed concurrently. After execution, each executor finalizes the writes generated by the execution to a peer called *storer*. The storer employs a Merkle tree [31] termed as the *state tree* to manage all states. In this paper, we focus on scaling execution with high flexibility. To this end, SChain separates storage from execution, making it simple for each organization to add or remove executors as needed.

B. Scalable concurrent execution

The scalability of concurrent execution comes from two levels: *intra-block* and *inter-block* concurrency.

Intra-block concurrency. The intra-block concurrency involves enabling multiple executors to collectively execute the transactions in every block concurrently. An executor E_i follows a concurrency control protocol to execute transactions in set B_h^i concurrently. After execution, the storer collects the execution results from all executors in the same organization for persistence. Following this scheme, a challenge arises accordingly: *the merge of execution results for $B_h^1, \ldots, B_h^m$ must be equivalent to the serial execution results for $\langle T_{h,1}, \ldots, T_{h,t} \rangle$ to meet the deterministic serializability.* In SChain, each executor E_i employs a *Deterministic Concurrency Control* (DCC) [50–55] protocol based on the *ordering lock* (Definition 2) to parallelize the execution for $\langle T_{h,p_1}, \ldots, T_{h,p_j} \rangle$.

Definition 2 (Ordering lock)**.** *Given a set of transactions $\langle T_{h,p_1}, \ldots, T_{h,p_j} \rangle$ in set B_h^i, for any pair of transactions T_{h,p_t} and T_{h,p_k} that both request lock on state r exclusively, then T_{h,p_t} must request the lock on r before T_{h,p_k} does if $p_t < p_k$.*

This approach may encounter an issue if there is a *cross-set dependency* between transactions $T_{h,i}$ and $T_{h,j}$ across different sets, where $T_{h,i}$ needs to read the writes produced by $T_{h,j}$. To overcome this, SChain allows executors to exchange intermediate writes to ensure the merge of partial results meets the deterministic serializability, as detailed in Section IV.

Inter-block concurrency. Albeit intra-block concurrency enables concurrent transaction execution within every block, the blocks are still processed sequentially. Some approaches attempt to improve resource utilization by pipelining the workflows of adjacent blocks, but the transaction execution is still performed on a block-by-block basis. More precisely, the transactions in block B_{h+1} cannot be executed before the execution for B_h completes even though some transactions in B_h and B_{h+1} can be executed in parallel. SChain, instead, terms transactions in $\langle \ldots, B_h, B_{h+1}, \ldots \rangle$ as a continuous transaction stream and keeps executors executing transactions concurrently, recognized as *inter-block concurrency*, to utilize capacity of all peers fully. As the execution of transactions in multiple blocks interleave, the generated writes may not arrive at storer as per the block order. Therefore, each storer in SChain groups the writes by block height and then commits them asynchronously.

IV. Intra-Block Concurrency

With an understanding of the basic idea of SChain, we present the intra-block concurrent execution in this section.

A. Graph-based block partition

At the beginning, the coordinator needs to partition $B_h = \langle T_{h,1}, \ldots, T_{h,t} \rangle$ into m disjoint sets $B_h^1, \ldots, B_h^m$ and forward them to m executors for concurrent execution. Ideally, it is desirable for each executor E_i to independently execute transactions in B_h^i with minimal interactions with other executors. It means fewer *cross-set dependencies*, as defined in Definition 3, exist between sets.

Definition 3 (Cross-set dependency)**.** *Given transactions $\langle T_{h,1}, \cdots, T_{h,t} \rangle$ in block B_h, if transaction $T_{h,i}$ is going to read state I[1], which will be written by $T_{h,j}$ with the greatest index j ($j < i$), a state dependency exists between $T_{h,i}$ and $T_{h,j}$. A cross-set dependency (CSD) $\mathcal{D}(T_{h,i}, T_{h,j}, I)$ arises if there is a state dependency between $T_{h,i}$ and $T_{h,j}$, which are in different sets.*

Due to the presence of a CSD $\mathcal{D}(T_{h,i}, T_{h,j}, I)$, $T_{h,i}$ should read the write on I generated by $T_{h,j}$. Thus, executor E_i that executes $T_{h,i}$ should transmit the write to E_j that executes $T_{h,j}$ (cf. Section IV-C). More CSDs will limit the degree of concurrency for execution, because executors need to wait for each other to transmit state values. Therefore, the block partition should: (1) make the size of all sets equal or close to achieve balanced load distribution. (2) minimize the number of CSDs to reduce the waiting time. SChain leverages a *forward graph* to model this problem.

Definition 4 (Forward graph)**.** *Given a block B_h, the forward graph of B_h is $G_h = \langle V, E \rangle$, where V and E are the sets of vertices and edges respectively. Each transaction is a vertex in $G_h.V$ and every state dependency between $T_{h,i}$ and $T_{h,j}$ on state I is an edge in $G_h.E$.*

Then, the block partition is translated into splitting the forward graph G_h into m subgraphs with close sizes (i.e., number of vertices), and minimizing the number of cut edges between them. This is a classic NP-hard problem in the graph mining field [56], thus SChain integrates the Metis library [57], including a series of optimized graph partitioning approximation algorithms, to split G_h. After splitting, the vertexes in each subgraph make up a set B_h^i.

B. Preprocessing

The coordinator in each organization needs to do some preparatory work to drive the concurrent execution on executors. The preprocessing for transactions in B_h comes with two steps: *information analysis* and *task distribution*. In the former step, the coordinator computes some necessary information to coordinate concurrent execution. Then, the coordinator

[1]In the absence of confusion, we use I to refer to the state item with key I.

distributes different tasks to executors and storer for execution in the later step by transmitting information to them.

Information analysis. As stated in Definition 2, the ordering lock adopted by SChain requires that the keys of state items to be accessed during execution should be obtained in advance. The coordinator has to know the *read/write set* for each transaction, which is detailed in Section VI. To ensure deterministic serializability, the executors exchange writes with each others during execution. Therefore, the coordinator includes some information in read/write set additionally to guide executors to transmit writes. The read/write set is formally defined in Definition 5.

Definition 5 (Read set)**.** *Given a transaction $T_{h,i} \in B_h^j$, its read set is $\mathcal{R}(T_{h,i}) = \{(I_t, O_t), \dots\}$, where I_t is the key of state to be read by $T_{h,i}$ and the parameter O_t indicates the source of the state value. The O_t can be: (1) '$\perp$', $T_{h,i}$ is going to read the value of I_t generated by another transaction $T_{h,k} \in B_h^i$ $(k < t)$; or (2)'$\swarrow$', the value of I_t is generated by a transaction not in B_h^i.*

All the transactions in set B_h^j will be executed by a same executor E. If O_t is '$\perp$', the value of I_t, to be read by $T_{h,i}$, is stored locally on E. Otherwise, if O_t is '$\swarrow$', it means the value of I_t will be written by transaction to be executed by another executor E', so that E should wait for E' to forward this value to drive the execution for $T_{h,i}$. The write set, as defined in Definition 6, reveals the forwarding relationship between transactions.

Definition 6 (Write set)**.** *The write set for transaction $T_{h,i}$ is $\mathcal{W}(T_{h,i}) = \{(I_t, L_t), \cdots\}$, where I_t is the key of state to be written by $T_{h,i}$. The L_t, namely* next list*, is formed as $\langle (T_{next}, E_{next}), \cdots \rangle$, where for every T_{next} (*next transaction*), there is a CSD $\mathcal{D}(T_{next}, T_{h,i}, I_t)$ and T_{next} is assigned to executor E_{next} (*next executor*).*

The next lists in write sets of all transactions document the topology of state values to be transferred among executors. When a $\mathcal{D}(T_{next}, T_{h,i}, I_t)$ emerges between E and E', E will forward value of I_t to E' with navigation of next list.

In SChain, executors request state values from storer to drive execution because there is no state data locally. To accelerate the execution phase, SChain lets the storer proactively forward state values to appropriate executors in advance. The coordinator determines a set, called *initial read set* including the states to be read from snapshot S_{h-1} to start execution phase for B_h.

Definition 7 (Initial read set)**.** *The initial read set for block B_h is $IR(B_h) = \{(I_t, E_{q_t}, T_{h,p_t}), \cdots\}$, where every state item I_t is in $\mathcal{R}(T_{h,p_t})$ and there is no transaction $T_{h,k}$ $(k < p_t)$ such that I_t is in $\mathcal{W}(T_{h,k})$. E_{q_t} is the executor assigned to execute T_{h,p_t}. Besides, we term $IR(B_h^i) = \{(I_t, T_{h,p_t}) | (I_t, E_{q_t}, T_{h,p_t}) \in IR(B_h) \wedge T_{h,p_t} \in B_h^i\}$ as the partial initial read set on B_h^i.*

$IR(B_h)$ contains all state values to be transmitted from storer to executors, which are the initial state data to drive

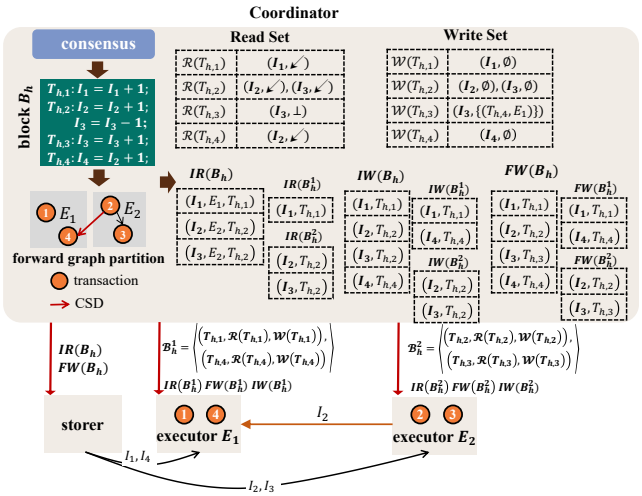


Fig. 4: Example of preprocessing for block B_h.

execution for B_h. After execution, each executor needs to flush results of execution to storer. Actually, only the state values in the *final write set*, as defined in Definition 8, should be sent to storer and other writes will be overwritten later. The storer can also decide whether the writes of block B_h are collected yet or not based on $FW(B_h)$.

Definition 8 (Final write set)**.** *The final write set for block B_h is $FW(B_h) = \{(I_t, T_{h,p_t}), \cdots\}$, where every I_t is in $\mathcal{W}(T_{h,p_t})$ and there is no transaction $T_{h,k}$ $(k > p_t)$ whose write set $\mathcal{W}(T_{h,k})$ contains I_t. The set $FW(B_h^i) = \{(I_t, T_{h,p_t}) \in FW(B_h) | T_{h,p_t} \in B_h^i\}$ is the partial final write set on B_h^i.*

Additionally, we can define the *initial write set* $IW(B_h)$ and *partial initial write set* $IW(B_h^i)$, which are similar to the definitions in Definition 7. The initial read set and initial write set are used to grant locks of states to transactions on every executor, which is detailed in Section IV-C.

The keys of states to be accessed by every transaction can be inferred through program analysis [58] or speculative execution [12] techniques. Then, the coordinator constructs the forward graph G_h for B_h, partitions it into m disjoint sets $\{B_h^1, \cdots, B_h^m\}$ and transforms them into $\{\mathcal{B}_h^1, \cdots, \mathcal{B}_h^m\}$ by filling read/write sets. In particular, the $\mathcal{B}_h^i$ is in the form

$$\mathcal{B}_h^i = \langle (T_h, p_1, \mathcal{R}(T_{h,p_1}), \mathcal{W}(T_{h,p_1})), \dots, \rangle$$

Last, the coordinator computes $IR(B_h)$, $IW(B_h)$ and $FW(B_h)$ for block B_h.

Task distribution. Once the information analysis completes, the coordinator notifies storer to forward state values to executors to drive execution by sending $IR(B_h)$ and $FW(B_h)$ to storer. Upon receiving $IR(B_h)$, storer forwards the values of states in $IR(B_h)$ to counterpart executors. Then, the coordinator sends the partial initial read set, partial final write set and set $\mathcal{B}_h^i$ to executor E_i. Last, every executor executes transactions concurrently based on received information.

Example 1. *Fig. 4 illustrates the preprocessing for four transactions in block B_h with two executors E_1 and E_2. First, the coordinator constructs the forward graph for B_h and*

partitions it into two disjoint equal-size sets (bottom-left part of Fig. 4). There is a CSD between two sets $B_h^1 = \{T_{h,1}, T_{h,4}\}$ (executed on E_1) and $B_h^2 = \{T_{h,2}, T_{h,3}\}$ (executed on E_2). Then, the coordinator fills the read/write set for every transaction. The symbol '$\swarrow$' in read set means the value of state should be forwarded by the storer (e.g., A in $\mathcal{R}(T_{h,1})$) or the other executor (e.g., B in $\mathcal{R}(T_{h,4})$). The coordinator further determines the $IR(B_h)$, $FW(B_h)$ and $IW(B_h)$. Next, the coordinator sends $IR(B_h)$ and $FW(B_h)$ to storer for early state value loading. When receiving $IR(B_h)$ and $FW(B_h)$, the storer forwards values of I_1, I_4 and I_2, I_3 to executors E_1 and E_2 respectively (buttom part of Fig. 4). Last, the coordinator dispatches $\mathcal{B}_h^1$, $IR(B_h^1)$, $FW(B_h^1)$ and $IW(B_h^1)$ to E_1, and sends $\mathcal{B}_h^2$, $IR(B_h^2)$, $FW(B_h^2)$ and $IW(B_h^2)$ to E_2. During the execution, executor E_2 should forward the write on I_2 generated by $T_{h,2}$ to drive the execution for $T_{h,4}$ on E_1.

C. Intra-block concurrent execution

Algorithm 1 presents the intra-block concurrent execution on an executor E_i. The algorithm takes a set of transactions $\mathcal{B}_h^i$, the partial initial read set $IR(B_h^i)$, the partial write set $IW(B_h^i)$, the partial final write set $FW(B_h^i)$, and a map W that buffers writes of transactions as inputs. To drive the execution, the values of states in the initial read set $IR(B_h^i)$ are loaded from the storer and stored in Ws on various executors initially. Then all state items to be read by transactions are ready locally on each executor. The execution of every transaction reads state values from W and writes updates to states back to it. After execution, the values of states in $FW(B_h^i)$ will be sent to storer for persistence.

First, for every (I_t, T_t) in $IR(B_h^i)$ and $IW(B_h^i)$, the executor E_i grants locks of I_t to T_t, and transactions that have collected all locks needed are inserted into queue Q_{active}, waiting for being executed (Line 1). This is because T_t is the first transaction in B_h accessing I_t. Second, execution for every transaction runs on a separate thread (Line 4). Third, for every (I_k, V_k) in the execution result R, E_i writes the value V_k to buffer W and send V_k to executors in next list L_k (Lines 5–9). The symbol '$\perp$' means value V_k is generated by a previous transaction on E_i (Line 6). Finally, the writes in W that are in the partial final write set $FW(B_h^i)$ are sent to the storer (Lines 10–12).

Upon receiving a write $\langle I_t, V_t, T_{h,p_t}\rangle$ from the other executor, E_i tries to write it to buffer W (Lines 13–17). E_i checks whether this write can be applied, i.e., every transaction $T_{h,p_k} \in \mathcal{B}_h^i$ ($p_k < p_t$) has finished its access to I_t (Line 15). Then, the value V_t of I_t is written into W (Line 16).

The details of $Write()$ is present in Lines 18–32. First, if the next transaction $\mathcal{T}_w$ accessing I_k is going to write I_k, E_i grants the lock of I_k to $\mathcal{T}_w$. At the same time, if $\mathcal{T}_w$ collects all locks, it is appended into Q_{active}, waiting for execution (Lines 19–23). Second, if there are multiple transactions that are going to read I_k, E_i grants read lock of I_k to these transactions, since multiple transactions can share a same read lock (Lines 25–32). Besides, the E_i should ensure the value source O_t of I_k matches the source O_{p_s} declared in read set of T_{h,p_s} (Line 29).

Algorithm 1: Intra-block concurrent execution on an executor E_i

Input: a set $\mathcal{B}_h^i$, partial initial read set $IR(B_h^i)$, partial initial write set $IW(B_h^i)$, partial final write set $FW(B_h^i)$, an instance of map[state] ⟨value, txn⟩ W that buffers writes of local transactions

```
1  Q_active ← Initialize(IR(B_h^i), IW(B_h^i))
2  for execution for ⟨T_{h,p_1}, ..., T_{h,p_j}⟩ not complete do
3      T_{h,p_t} ← Q_active.pop()
       // Run on a separate thread
4      R ← execute_txn(T_{h,p_t}, W)
5      foreach (I_k, V_k) ∈ R do
6          Write(W, I_k, V_k, T_{h,p_t}, ⊥)
7          L_k ← Next_List(W(T_{h,p_t}), I_k)
8          foreach (T_next, E_next) ∈ L_k do
9              send ⟨I_k, V_k, T_next⟩ to executor E_next
10 foreach (I_t, ⟨V_t, T_{h,p_t}⟩) ∈ W do
11     if (I_t, T_{h,p_t}) ∈ FW(B_h^i) then
12         send ⟨I_t, V_t, T_{h,p_t}⟩ to storer
13 event receive ⟨I_t, V_t, T_{h,p_t}⟩ from other executor do
14     for true do
15         if Can_Write(I_t, T_{h,p_t}) is true then
16             Write(W, I_t, V_t, T_{h,p_t}, ↙)
17             Break
18 procedure Write(W, I_k, V_k, T_{h,p_t}, O_t) do
19     T_w ← Next_Write_Txn(I_k, T_{h,p_t})
20     if T_w ≠ null then
21         Grant_Write_Lock(I_k, T_w)
22         if T_w collect all locks then
23             Q_active.push(T_w)
24     else
25         T_r ← Next_Read_Txns(I_k, T_{h,p_t})
26         if T_r ≠ null then
27             foreach T_{h,p_s} ∈ T_r do
28                 O_{p_s} ← Get_Read_Source(T_{h,p_s}, I_k)
29                 if O_{p_s} = O_t then
30                     Grant_Read_Lock(I_k, T_{h,p_s})
31                     if T_{h,p_s} collect all locks then
32                         Q_active.push(T_{h,p_s})
```

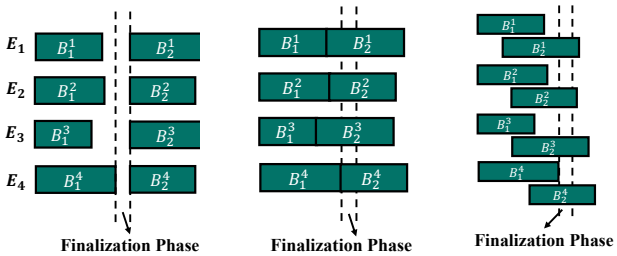
Similarly, if T_{h,p_s} collects all locks needed yet, it will be inserted into Q_{active} as well. After collecting all writes of transactions in block B_h, the storer applies these to state tree and make a new snapshot S_h of global state. The transactions in block B_{h+1} can read state values from S_h.

V. Inter-Block Concurrency

The intra-block concurrency merely enables concurrent execution for transactions within every block. In this section, we further explore the concurrency across blocks.

A. Inter-block execution model

In traditional blockchains, the entire workflow proceeds in a synchronous manner [1, 2, 59], as exemplified in Fig. 5(a), where the workflow for block B_{h+1} cannot start before that for block B_h terminates. Fig. 5(b) pipelines the workflows for multiple blocks. Then each executor can proceed to execute transactions in block B_2 once execution for block B_1 is finished,



(a) Sync (b) Partial sync (c) Inter-block

Fig. 5: Comparison between execution models.

and finalizes B_1 asynchronously. However, this execution model may not fully exploit the capacity of every executor since the execution for blocks is performed serially. So we term it as a *partial sync* execution model. For example, the execution for transactions in B_1^1 are bound to four worker threads concurrently on executor E_1. At the end of execution phase for B_1^1, most threads may stay idle and wait for the rest one to execute remained transactions that cannot be executed in parallel.

Instead, SChain allows threads to execute transactions in successive blocks, as long as they do not read any writes to be generated by unexecuted transactions as shown in Fig. 5(c), recognized as an *inter-block* execution model. To ensure deterministic serialazibility, executors should promise that the execution for every transaction always reads the correct value of each state, i.e., the write produced by transaction proceeding itself. To this end, each executor sends the generated writes to the storer in real time, so that other executors can read the latest value of each state to drive the execution.

B. Sliding window-based cache

With the inter-block execution model, the order that writes arrive at the storer may not match the transaction order. If storer applies every write to the state tree once arrival, the effect of these transactions may not be serializable. To address this, the storer leverages a *sliding window-based cache* (SWC) to isolate the writes of different blocks and manages them independently. The top of Fig. 6 depicts an instance of SWC, which sets two cache slots C_{h+1} and C_{h+2} to buffer the writes generated by transactions in blocks B_{h+1} and B_{h+2}, respectively. When receiving a write, storer adds it to counterpart cache slot. Specifically, the cache slot C_h can be in one of three different states as follows.

- **Unready:** the writes, i.e., the final write set $FW(B_h)$, at block height h have not been collected in C_h.
- **Ready:** all writes in $FW(B_h)$ have arrived at C_h.
- **Finalized:** the cache C_h is ready, snapshot S_l has been made by storer and the root hash of state tree is available.

The storer finalizes the writes in multiple cache slots as per block order to ensure the serializability and determinism. With a ready cache slot C_h, the storer finalizes it asynchronously, not blocking the workflows of other blocks. To check whether C_h is ready, the final write set $FW(B_h)$ is sent to the storer. If so, the storer applies writes in C_h to state tree. Since the writes of different blocks are isolated, the storer can still add writes

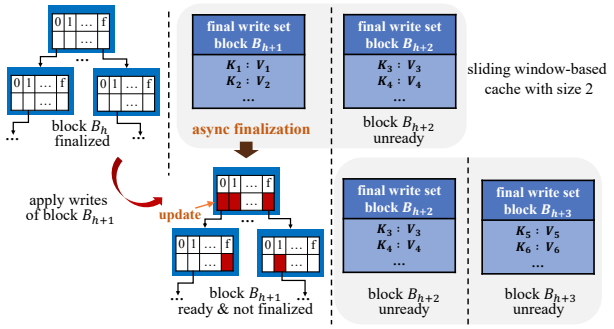


Fig. 6: Example of sliding window-based cache for blocks B_{h+1} and B_{h+2} with size $\delta = 2$. The cache for B_{h+1} is finalized asynchronously and the window slides to block B_{h+3}.

Algorithm 2: Initial read set reading

Input: $IR(B_h)$:initial read set for block B_h, h: specified block height

1 **for each** $(I_t, E_{q_t}, T_{h,p_t})$ **in** $IR(B_h)$ **do**
2 **for** $l \leftarrow h-1$ to 0 **do**
3 **if** $finalized(C_l)$ is true **then**
4 $V_t \leftarrow read_smt(I_t, l)$
5 send $\langle I_t, V_t, T_{h,p_t} \rangle$ to executor E_{q_t}
6 **break**
7 **if** I_t **is in** $FW(B_l)$ **then**
8 **if** I_t **is in** C_l **then**
9 $V_t \leftarrow read_cache(I_t, C_l)$
10 send $\langle I_t, V_t, T_{h,p_t} \rangle$ to executor E_{q_t}
11 **else**
12 $add_wait(l, I_t, T_{h,p_t}, E_{q_t})$
13 **break**

14 **event** receive $\langle I_t, V_t, T_{l,p_s} \rangle$ from executor **do**
15 $C_l.insert(I_t, V_t)$
16 $wait \leftarrow get_wait(l, I_t)$
17 **foreach** $E_{q_k}, T_{h,p_k} \in wait$ **do**
18 send $\langle I_t, V_t, T_{h,p_k} \rangle$ to executor E_{q_k}

of other blocks to SWC even during the finalization for C_h. Once the cache slot C_h is finalized, the state tree reflects the latest value of every state till block height h. SChain specifies a parameter δ to limit the number of unready cache slots to avoid the size of SWC grows infinitely. If the latest finalized slot is C_h, coordinator cannot schedule the execution for any transaction in block B_k such that $k > h + \delta$. To some extent, the cache slots function as a sliding window over blocks.

State reading. The SWC makes the reading for states in $IR(B_h)$ more complicated. When the storer processes an item $(I_t, E_{q_t}, T_{h,p_t})$ in $IR(B_h)$, it cannot load value of I_t from state tree directly if cache slot C_{h-1} is not finalized. Instead, the storer searches every C_j $(j < h)$ forward to determine the latest value of I_t. Algorithm 2 presents the read procedure for $IR(B_h)$ on the storer. For each $(I_t, E_{q_t}, T_{h,p_t})$ in $IR(B_h)$, storer searches it from the latest block height $h-1$ (Lines 2–13). At each height l, if C_l is finalized, the storer can access the latest value of I_t from state tree because it reflects latest values of all states till B_l (Lines 3–6). Otherwise, the storer checks whether I_t is in $FW(B_l)$ for block B_l. If not so, I_t is

Algorithm 3: Inter-block concurrent execution on an executor E_i

```
Data: a queue Q_active of active transactions, an instance of
      map[state] ⟨value, txn⟩ W, an instance of map[block height]
      ⟨partial final write set⟩ F
1  for true do
2      T_{h,p_t} ← Q_active.pop()
       // Run on a separate thread
3      R ← execute_txn(T_{h,p_t}, W)
4      foreach (I_k, V_k) ∈ R do
5          Write(W, I_k, V_k, T_{h,p_t}, ⊥)
6          L_k ← Next_List(W(T_{h,p_t}), I_k)
7          foreach (T_next, E_next) ∈ L_k do
8              send ⟨I_k, V_k, T_next⟩ to executor E_next
9              if (I_k, T_{h,p_t}) ∈ F[h] then
10                 send ⟨I_t, V_t, T_{h,p_t}⟩ to storer
11 event receive ⟨B_h^i, FW(B_h^i)⟩ from coordinator do
12     store B_h^i locally
13     F[h] ← FW(B_h^i)
14 event receive ⟨I_t, V_t, T_{h,p_t}⟩ from other executor or storer do
15     same as Lines 14-17 in Algorithm 1
```

not updated at block height l and storer skips to next block height $l-1$. Otherwise, the latest value for I_t should be in C_l (Lines 8–10). If the value of I_t has not arrived at C_l , storer marks that this value is awaited by transaction T_{h,p_t} through function $add_wait()$ (Line 12). Once receiving the value of I_t, storer forwards it to counterpart executors waiting for it (Lines 14–18).

C. Inter-block concurrent execution

Algorithm 3 presents the framework of inter-block concurrent execution with an asynchronous workflow on an executor E_i. There are three global data in this algorithm, which are shared across executor E_i. The Q_{active} is the queue of active transactions to be executed. The W is a cache to buffer values of states and $\mathcal{F}$ maps the block height to the partial write set at that height. The main loop continues to retrieve active transaction from Q_{active} and bind it to a thread for concurrent execution (Lines 1–10). Different from Algorithm 1, when the execution for a transaction completes, the executor tries to send its writes to the storer immediately to enable other transaction can read them earlier (Lines 9–10). In particular, the $\mathcal{F}$ will be updated later at Line 13. When receiving the assigned task (i.e., $\langle \mathcal{B}_h^i, FW(B_h^i)\rangle$ from the coordinator, the executor E_i adds it to set $\mathcal{F}$. The $\mathcal{F}$ is used to check whether the writes of a transaction are in the final write set, which should be sent to the storer (Lines 11–13). Note that the coordinator does not have to send sets $IR(B_h^i)$ and $IW(B_h^i)$ to every executor E_i, because there is no need to initialize Q_{active} based on $IR(B_h^i)$ and $IW(B_h^i)$. Instead, E_i manages the values sent from other executors and storer universally. When E_i receives a value item $\langle I_t, V_t, T_{h,p_t}\rangle$, it tries to apply it to W and active some transactions, which would be scheduled for execution (Lines 14–15).

As can be seen, the execution for transactions in different blocks interleave. A transaction in a later block can be executed earlier if its read set is ready, which enlarges the parallelism of execution. Besides, the asychronous finalization will not block the concurrent execution. During the finalization, executors can still insert writes to SWC. Therefore, the entire workflow becomes asynchronous and the computation resources of multiple threads of an executor can be utilized fully.

D. Scalability & flexibility

The decoupling design of SChain exhibits notable ease to scale different services. We discuss how to scale the *ordering*, *execution* and *finalization* respectively. This paper primarily focuses on scaling the execution, whereas existing proposals can be leveraged to scale the ordering and finalization services.

Ordering. The throughput of the ordering service should match the capability of concurrent execution in SChain. However, a single PBFT instance, as the most popular consensus implementation in permissioned blockchains, merely handles thousands of transactions per second, far from the demanded throughput. Fortunately, many advanced consensus protocols [60–62] have been proposed recently, and we can adopt these protocols to achieve higher performance. Besides, we can directly add more instances of PBFT and then merge the produced results of them to improve the throughput.

Execution. When the execution rate of executors within an organization fails to catch up the transaction rate of ordering, the organization can add more executors to scale the execution. Since every executor is stateless, an organization can add/remove executors without data migration. To add an executor, a new executor sends a notification message to the coordinator first, then the coordinator proceeds to allocate tasks to it during the coordination for the next block. The removal of an executor differs slightly. The executor to be removed should execute all assigned transactions before quitting to avoid incomplete execution. Despite not persisting any state data, executors are required to preserve the codes of invoked smart contracts. This is because a transaction merely carries the arguments to a function of the called smart contract. In SChain, a new executor does not need to synchronize codes of all smart contracts. Instead, the executor just requests the code of a smart contract from the storer when processing a transaction invoking it. Notably, every organization can scale the execution independently, without the need to communicate with other organizations. This design significantly enhances the flexibility to scale the execution service.

Finalization. The current design of SChain utilizes a single storer within each organization for finalization since it can effectively serve the execution. Actually, when the storer becomes a bottleneck, SChain can employ the *distributed state trees* managed by multiple storers to handle the state data. Additional details regarding this approach can be referred to [63]. Based on this approach, the state data is partitioned across all storers and the coordinator should distribute the state reading and writing tasks over them accordingly.

E. Transaction aborting

In SChain, a user should analyze the keys of states to be accessed by each transaction and attach them to the transaction

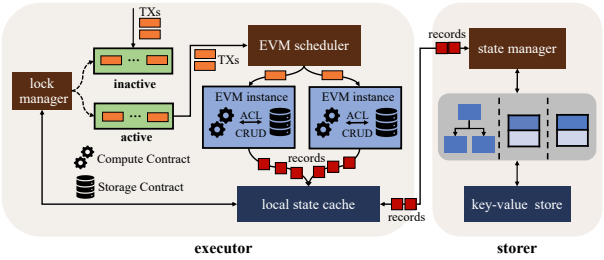


Fig. 7: The implementation of the executor and the storer.

(the state values are not required). During the execution phase, if the transaction attempts to access a state that was not declared in advance, it will be aborted, and the user needs to analyze and resubmit it. The details of transaction aborting can be found at [64].

VI. Implementation and Discussion

In this section, we present the implementation of SChain.

A. Implementation of three phases

We leverage two schemes to scale the ordering service in our experiments.

Multiple PBFT instances. We run multiple PBFT instances on multiple groups of nodes. Each group runs a PBFT instance to establish a partial order of transactions packed as a *micro block* and sends it to the coordinator. Then the coordinator merges all micro blocks into a normal block according to a deterministic rule.

Parallel consensus. Some works [60–62] parallelize the consensus with the leaderless design. In these works, nodes can run agreement on multiple blocks simultaneously. We integrate the RCC [60] as an implementation of our ordering service and recommend readers to refer to [60] for details.

Fig. 7 shows the implementation of the executor and the storer. SChain integrates the Ethereum Virtual Machine (EVM) [2] as the running environment for transactions. On each executor, every transaction is placed in an inactive transaction queue first and then lifted to the active transaction queue. Besides, there is a lock manager, an EVM scheduler and a state cache. The state cache is the implementation of W in Algorithm 1, which is used to cache state values to be accessed during execution. Since the original EVM does not support multiple-thread programs, executor builds an EVM pool to accommodate multiple EVM instances initially. Once a transaction becomes active, the EVM scheduler binds it to an idle EVM instance and assigns a thread for execution. On the storer, the state manager is in charge of inserting writes to SWC or state tree and reading the states in each initial read set. The storer leverages a key-value store RocksDB to persist the state tree, where each node in state tree is treated as a separate record in RocksDB.

B. Smart contract

Smart contract classifying. SChain splits the functionality of a smart contract into the *storage contract* and the *computing contract*. The former provides a storage service with CRUD interfaces (create, read, update and delete) that are implemented as four standard functions. The later is the real logic of applications, which reads/writes data from/to the storage contract via provided CRUD interfaces. The storage contract is similar to the table in databases, while the computing contract is like the stored procedure. This decoupling is very useful for lock management and contract upgrading.

Read/write set detection. SChain employs a static code analysis tool, Slither [58] designed for Solidity, to estimate the read/write set for each function of a smart contract. In particular, Slither generates the *control-flow graph* (CFG) [65] for every smart contract, recognizes all nodes in CFG that perform read or write operations, and infers concrete keys of states to be accessed. If the keys cannot be solved statically, we execute the contract code (only a forward slice of the contract code is executed in our implementation) upon the input of a transaction and values of states from a committed snapshot of state tree. In practice, a client can perform such analysis locally or delegate it to an organization. Then the client submits this transaction together with the read/write set to SChain. If a malicious organization returns incorrect read/write set, it doesn't affect the correctness because a transaction that tries to access states not declared in its read/write set will be aborted.

C. Recovery and attacks

In this subsection, we discuss how to recover the functionality of SChain in case of faults and defend against potential attacks.

Ordering. If peers in the ordering service fail, an organization cannot obtain transactions to execute, thus it replaces these peers with new ones. When the ordering service recovers, the peers will synchronize missed transactions from other organizations and then execute them to catch up. To accelerate the execution, organization can add some additional executors and removes them when catch up.

Coordinator. If the coordinator crashes, its organization will replace it with a new one. Then, the new coordinator replays transactions in blocks that are not committed yet. If B_h is the latest committed block, the transactions in blocks after B_h should be re-executed, because some executors may not have received tasks for these blocks. To this end, all executors halt their local execution, after which the new coordinator coordinates the execution as usual. The storer does not need to clear the cache slots for blocks after B_h, because duplicated writes have no impact on the consistency of final results.

Executor. If an executor fails, the storer cannot collect the writes that should be produced by this executor. Similarly, the organization adds a new executor to replace the crashed one and notify the coordinator to replay transactions in blocks after B_h, the latest committed block. Surely, the organization can simply drop this crashed executor and let remained executors to execute transactions.

Storer. When the storer becomes crashed, the organization can restart it or replace it with a new one. If the state tree remains available, this organization can replay transactions

TABLE I: Experiment parameters

Parameter	Value	Parameter	Value
# of worker threads	1 2 3 4	# of executors	2 4 6 8 10
workload types	UW SW	hot ratio (%)	10 30 50 70 90
graph partition	Rand Metis	# of accounts	1M

after latest committed block as described above. If not so, the new storer synchronizes a snapshot S_h of the state tree. Then the coordinator replays transactions after block height h to catch up quickly. Meanwhile, the storer can synchronize missed snapshots from other organizations lazily.

Attacks. In SChain, we regard an organization malicious as if one of its peers behaves maliciously. A malicious organization can try to send incorrect tasks and wrong values of states to executors or storers in other organizations, forward conflicting blocks to other coordinators, or prevent system from achieving agreement on blocks. However, the coordinator, executors and storer just trust peers from the same organization and ignore messages sent from other organizations. The only attack that malicious organizations can conduct is to break the ordering service, which is solved by the consensus protocols.

D. Correctness

We have proven the correctness of our proposed concurrent execution. First, we prove the deterministic serializability for every block B_h as described in Theorem 1. The basic idea is to prove that our proposed scheme does not violate the ordering lock scheme as presented in Definition 2 and the details to this theorem can be found at [64].

Theorem 1. *With intra-block concurrency, the effect of writes produced by all executors for B_h is equivalent to the serial order predefined by B_h.*

Second, we explain the inter-block concurrent execution still remains correct for SChain. To drive the execution for transactions in current block, the coordinator notifies the storer to load relevant states to each executor, rather than allowing transactions from previous blocks to forward their writes to transactions. Besides, the finalization phases for different blocks are performed sequentially. Therefore, if we can ensure the execution for each block is deterministically serializable, the execution across blocks still guarantees this property.

VII. Experimental Evaluation

A. Setup and workloads

Testbed. In all experiments, we deploy up to four organizations, each of which includes up to 10 executors. In each test, we ensure the throughput of the ordering service matches the performance of the execution by scaling the ordering service. We measure the latency of transactions when the throughput no longer increases with more clients. The RocksDB, a key-value store, is used to persist the state tree, where each node of the state tree is stored as a separate record. Each peer runs on a physical machine with Intel Xeon E5-2620 2.1GHz CPU and 16GB RAM. The machines are connected via 10Gb Ethernet.

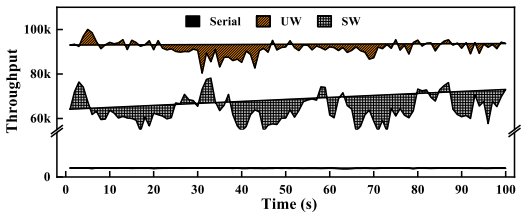


Fig. 8: Throughput of SChain over time.

Workload. We use the Smallbank benchmark adopted by many researches [17, 18, 66] for our tests. In particular, there are 1M bank accounts in total and every transaction reads and writes four bank accounts, respectively. We set two types of workloads in the evaluation: uniform workload (**UW**) and skewed workload (**SW**). In the former workload, every transaction accesses all accounts evenly. In the latter workload, we manually set 10% of the accounts to hot accounts. Each read/write has a certain probability to access the hot accounts, controlled by the *hot ratio* parameter. With greater hot ratio, there are more conflicts between transactions. Table I tabulates all parameters, with the default value underlined.

Comparison. We compare the performance of our inter-block execution model with other two models depicted in Fig. 5. Besides, SChain is compared with some competing systems-level approaches, including Fabric 2.0 with multiple-channel, Fabric# [67], AHL [24], RingBFT [42] and SSI, a concurrent execution model used in [14]. For a fair comparison, we implement SSI in the decoupling architecture of SChain.

B. Performance of SChain

In this subsection, we evaluate the performance of SChain under different settings.

Overall performance. We evaluate the overall performance of SChain. Fig. 8 shows the throughput of SChain under uniform and skewed workloads over time respectively with 10 executors per organization. We also implement a basic blockchain with the serial execution manner (the `Serial`) as the baseline. For the `UW` and `SW`, we count the throughput of all 10 executors. Since transactions are processed by a single thread of a peer, the performance of `Serial` will not be affected by the type of workload, thus we just report the results under `SW`. The throughput of SChain under `SW` becomes lower compared with that under `UW` since there are more conflicts between transactions and the parallelism is low. Fig. 9 reports the throughput of four out of the 10 executors over time. Each executor achieves an average throughput about 8.5K TPS, 2.12× of the `Serial` and the overall throughput of 10 executors is about 20 × higher than the `Serial`. Due to the inter-block concurrent execution, every executor remains consistently utilized.

Hot ratio. Fig. 10 reports the throughput and latency of SChain with different hot ratios under the default configuration. We also implement the *random partition strategy* as a comparison, in which transactions of a block are assigned to all executors in random. Fig. 10(a) shows the throughput of SChain with two strategies termed as `Random` and `Metis`. As the hot

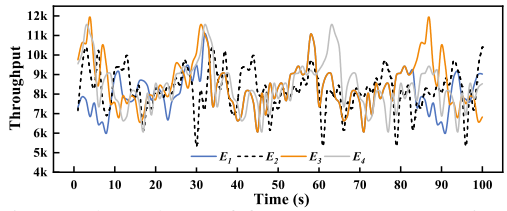
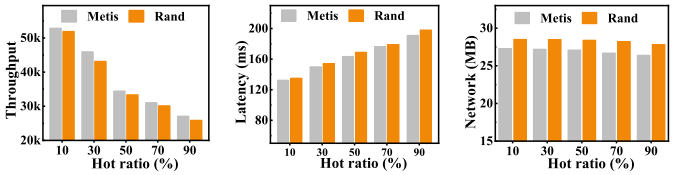


Fig. 9: Throughput of four executors over time.



(a) Throughput (b) Latency (c) Network

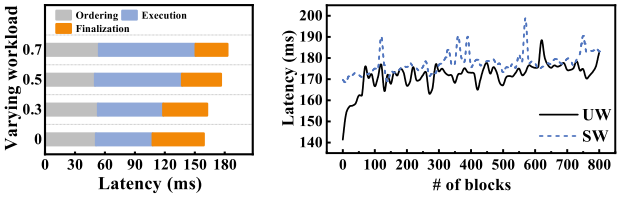
Fig. 10: Performance of SChain under varying hot ratio.

ratio increases, the throughput for both strategies decreases accordingly. The throughput of SChain decreases from 34K to 25K as the hot ratio varies from 10% to 90%. The the partition for both approaches spends less 1ms. The `Metis` achieves better results than `Rand` does, but the advantage is not significant. When the hot ratio is low, the parallelism between transactions is high, so both approaches perform good in block partition. When the hot ratio becomes higher, even though `Metis` can reduce the number of CSDs, the parallelism between transactions decreases and many intermediate state values need to be exchanged, which is the main overhead to performance. We measure the latency of transactions by varying the hot ratio in Fig. 10(b). The latency just counts the time from when the client sends a transaction to when it receives the same reply from a majority of organizations, excluding the latency for finalization. Similarly, the latency for both strategies increases as the hot ratio grows, which is a direct result of throughput decline. Then we summarize the network consumption within an organization per block, as shown in Fig. 10(c). Since the transmission of the read/write set dominates the network (over 90%), the advantage of `Metis` is not significant as well.

Latency of various phases. We evaluate the overhead of three phases in Fig. 11. Fig. 11(a) analyzes the latency of three phases per block with four hot ratios, where 0 represents uniform workload and the rest represent skewed workloads (Y-axes are 0.3, 0.5 and 0.7 respectively). As the hot ratio grows from 0 to 0.7, the latency for ordering phases remains stable (48ms). The latency for execution phase varies from 50ms to 100ms. This supports that a greater hot ratio limits the parallelism between transactions. On the contrary to our expectation, the latency for finalization may decrease as the hot ratio increases, since a greater hot ratio means fewer states should be updated. Then, fewer nodes in the state tree are modified, resulting in lower latency. Fig. 11(b) details the latency over blocks. Note that the latency may suddenly increase. This is caused by the *compaction* of RocksDB during writing.

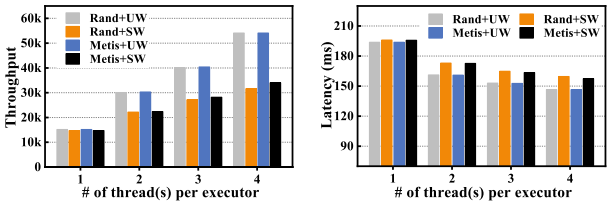
C. Scalability

In this subsection, we test the scalability of SChain.



(a) Latency (b) Latency over blocks

Fig. 11: Latency of three phases.



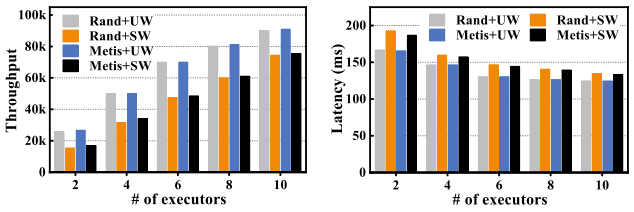
(a) Throughput (b) Latency

Fig. 12: Performance under varying number of threads.

Number of threads. Fig. 12(a) reports the throughput of four executors against the number of threads per executor varies from 1 to 4. With the growth of thread number, the throughput under all settings increases accordingly. When the number of threads reaches 4, the throughput is improved by a factor near 4 under `UW`, while the factor is up to 2.8 under `SW`. There are much fewer conflicts between transactions under `UW`, thus SChain can scale the execution better. Fig. 12(b) reports the latency of transactions. With more threads, the latency declines since the latency of execution is reduced.

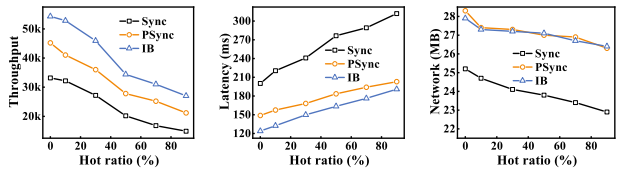
Number of executors. Fig. 13(a) reports the sum of up to 10 executors' throughput. The overall throughput of SChain is proportional to the number of executors, up to 90K TPS with 10 executors. Similarly, SChain scales better under `UW`. Fig. 13(b) depicts the latency of SChain under different settings. The latency for `Rand+SW` decreases from 250ms to 160ms as the number of executors varies from 2 to 10.

Various execution models. We evaluate SChain with three execution models, as shown in Fig. 5: (i) synchronous (`Sync`), (ii) partial synchronous (`PSync`) and (iii) inter-block (`IB`). Fig. 14 reports the performance of all models against varying hot ratio. The hot ratio with 0 is a uniform workload. The throughput of `IB` approach is twice that of `Sync`, which achieves only 14K TPS with hot ratio 0.9, because the finalization in `Sync` blocks the entire workflow. Even though `PSync` removes the blockage of finalization, all threads on single executor are synchronized at the end of execution for every block. Fig. 14(b) shows the latency of various models. The latency of `Sync` is about twice of `PSync` and `IB`. For example, when the hot ratio is 90%, the latency for `Sync` and `PSync` are 310ms and 195ms respectively. Fig. 14(c) reports the network consumption per block within an organization. The `Sync` occupies a smaller amount of network bandwidth (<25MB). This is because `Sync` sends all writes to the storer in a batch, while `PSync` and `IB` send the every write in final write set in real time.



(a) Throughput (b) Latency

Fig. 13: Performance under varying number of executors.



(a) Throughput (b) Latency (c) Network

Fig. 14: Performance of varying execution models.

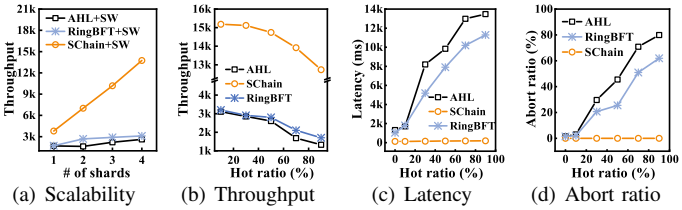
D. Comparison with other approaches

Last, we compare SChain with other approaches designed to scale the performance of the blockchain.

Comparison with sharding. To verify the scalability of SChain, we compare it with sharded systems AHL and RingBFT. We implement a prototype of the sharded blockchain and apply the protocols to it. In `AHL` or `RingBFT`, each shard includes 4 peers and up to 4 shards are deployed in our tests. The state data are partitioned over all shards evenly. Within each shard, all peers perform PBFT algorithm for consensus of transactions. AHL [24] uses a 2PC-based protocol to handle distributed transactions (CSTs) and RingBFT [42] processes CSTs via a ring-based communication, where a CST can be finished within two rings of communications. Since each shard executes transactions sequentially, each executor in `SChain` executes transactions sequentially in all tests for a fair comparison. Fig. 15(a) shows the throughput of all systems with varying number of shards (or executors)[2]. As can be seen, both sharded systems merely reach a throughput less than 3,000 with four shards, because the CSTs account for a large percentage, severely limiting the scalability. We then test the throughput of both systems with varying hot ratio in Fig. 15(b). As the hot ratio increases, the throughput of `AHL` and `RingBFT` decline dramatically. The throughput is even close to 1K with four shards when the hot ratio is 90%. Fig. 15(c) presents the latency of all systems. As the hot ratio increases, `SChain` always remains a low latency while the latency of `AHL` and `RingBFT` grows up to 14s when the hot ratio reaches 90%. This is because with a high hot ratio, high competition incurs most transactions aborted [24, 68] as shown in Fig. 15(d). Therefore, sharding cannot scale well in the presence of CSTs.

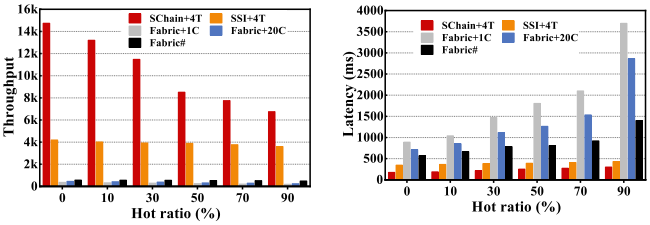
Comparison with Fabric, Fabric# and SSI . We compare SChain with other concurrency controls like `SSI` [14] and `Fabric#` [17]. Besides, we also compare SChain with Fabric 2.0 with the multi-channel design. We implement `SSI` in

[2]Every organization in SChain adds an executor is equivalent to that `AHL` and `RingBFT` add a shards.



(a) Scalability (b) Throughput (c) Latency (d) Abort ratio

Fig. 15: Comparison with sharding.



(a) Throughput (b) Latency

Fig. 16: Comparison with SSI, Fabric 2.0 and Fabric#.

the decomposed architecture of SChain. In `SSI`, transactions read state values from the latest finalized snapshot first, and then the transactions that violate deterministic serializability are aborted and re-executed till no one to be aborted [14]. Since `SSI` cannot be applied with multiple executors directly, all tests are conducted with one executor. We test Fabric 2.0 with 1 and 20 channels respectively. The architecture of `Fabric#` is different from SChain, thus we just use its original project [67] for tests. Fig. 16(a) reports the throughput of various systems. `Fabric+1C` achieves the lowest throughput (about 200 TPS) since the validation for each transaction is performed sequentially. Although transactions belongs to multiple channels can be processed in parallel, the improvement of `Fabric+20C` is not significant. The throughput of `SSI` achieves 5,800 with 4 threads per node. However, `SChain` still outperforms `SSI`, because `SSI` adopts a partial synchronous execution model. Instead, `SChain` implements the inter-block concurrent execution, allowing transactions in multiple blocks to be executed concurrently. Fig. 16(b) shows the latency of various systems. As expected, `SChain` outperforms others significantly due to its high degree of concurrency.

VIII. Conclusion

Scalability and flexibility plays a crucial role in financial blockchain applications. In this paper, we propose SChain, a flexible blockchain system, achieving scalable concurrent execution from intra- and inter-block concurrency. The intra-block concurrency extends single-point concurrent execution to multiple executors while promising deterministic serializability. The inter-block concurrency explores concurrent execution between transactions across blocks, resulting in higher parallelism, i.e., more transactions can be executed in parallel, due to full utilization of resources from various peers.

Acknowledgments

This work was supported by National Key Research and Development Program of China (No. 2021YFB2700100) and National Science Foundation of China (No. 61972152 and No. U1911203). Zhao Zhang is the corresponding author.

REFERENCES

[1] S. Nakamoto *et al.*, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[2] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.

[3] I. Eyal, A. E. Gencer, E. G. Sirer, and R. Van Renesse, "Bitcoin-ng: A scalable blockchain protocol," in *13th {USENIX} symposium on networked systems design and implementation ({NSDI} 16)*, 2016, pp. 45–59.

[4] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proceedings of the thirteenth EuroSys conference*, 2018, pp. 1–15.

[5] "Quorum," https://consensys.net/quorum/, [n.d.].

[6] "Diem," https://www.diem.com/, [n.d.].

[7] K. Korpela, J. Hallikas, and T. Dahlberg, "Digital supply chain transformation toward blockchain integration," in *proceedings of the 50th Hawaii international conference on system sciences*, 2017.

[8] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, "Medrec: Using blockchain for medical data access and permission management," in *2016 2nd international conference on open and big data (OBD)*. IEEE, 2016, pp. 25–30.

[9] J. Dean and S. Ghemawat, "Mapreduce: simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.

[10] P. C. A. Katsifodimos, S. E. V. Markl, and S. H. K. Tzoumas, "Apache flinktm: Stream and batch processing in a single engine," *Bull. IEEE Comput. Soc. Tech. Commun. Data Eng.*, vol. 36, no. 4, 2015.

[11] "Blockchain-as-a-Service," https://www.alibabacloud.com/product/baas, 2022.

[12] M. J. Amiri, D. Agrawal, and A. El Abbadi, "Parblockchain: Leveraging transaction parallelism in permissioned blockchain systems," in *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2019, pp. 1337–1347.

[13] C. Jin, S. Pang, X. Qi, Z. Zhang, and A. Zhou, "A high performance concurrency protocol for smart contracts of permissioned blockchain," *IEEE Transactions on Knowledge and Data Engineering*, 2021.

[14] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran, "Blockchain meets database: Design and implementation of a blockchain relational database," *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1539–1552, 2019.

[15] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen, "Adding concurrency to smart contracts," *Distributed Computing*, pp. 1–17, 2019.

[16] S. Pang, X. Qi, Z. Zhang, C. Jin, and A. Zhou, "Concurrency protocol aiming at high performance of execution and replay for smart contracts," *arXiv preprint arXiv:1905.07169*, 2019.

[17] P. Ruan, D. Loghin, Q.-T. Ta, M. Zhang, G. Chen, and B. C. Ooi, "A transactional perspective on execute-order-validate blockchains," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 543–557.

[18] A. Sharma, F. M. Schuhknecht, D. Agrawal, and J. Dittrich, "Blurring the lines between blockchains and database systems: the case of hyperledger fabric," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 105–122.

[19] Z. Peng, Y. Zhang, Q. Xu, H. Liu, Y. Gao, X. Li, and G. Yu, "Neuchain: A fast permissioned blockchain system with deterministic ordering," *transactions*, vol. 14, no. 30, pp. 52–55.

[20] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX NSDI*, 2019, pp. 95–112.

[21] J. Hellings and M. Sadoghi, "Byshard: Sharding in a byzantine environment," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2230–2243, 2021.

[22] M. J. Amiri, D. Agrawal, and A. El Abbadi, "Sharper: Sharding permissioned blockchains over network clusters," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 76–88.

[23] S. Das, V. Krishnan, and L. Ren, "Efficient cross-shard transaction execution in sharded blockchains," *arXiv preprint arXiv:2007.14521*, 2020.

[24] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proceedings of the 2019 international conference on management of data*, 2019, pp. 123–140.

[25] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 583–598.

[26] X. Qi, "S-store: A scalable data store towards permissioned blockchain sharding," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1978–1987.

[27] P. Zheng, Q. Xu, Z. Zheng, Z. Zhou, Y. Yan, and H. Zhang, "Meepo: Sharded consortium blockchain," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 1847–1852.

[28] Z. Chen, H. Zhuo, Q. Xu, X. Qi, C. Zhu, Z. Zhang, C. Jin, A. Zhou, Y. Yan, and H. Zhang, "Schain: a scalable consortium blockchain exploiting intra- and inter-block concurrency," *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 2799–2802, 2021.

[29] Z. Xu, S. Han, and L. Chen, "Cub, a consensus unit-based storage scheme for blockchain system," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 173–184.

[30] "AntChain," https://antchain.antgroup.com/, 2020.

[31] R. C. Merkle, "A certified digital signature," in *Conference on the Theory and Application of Cryptology*. Springer, 1989, pp. 218–238.

[32] L. Baird, "The swirlds hashgraph consensus algorithm: Fair, fast, byzantine fault tolerance," *Swirlds Tech Reports SWIRLDS-TR-2016-01, Tech. Rep*, vol. 34, 2016.

[33] C. Li, P. Li, D. Zhou, Z. Yang, M. Wu, G. Yang, W. Xu, F. Long, and A. C.-C. Yao, "A decentralized blockchain with high throughput and fast confirmation," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 515–528.

[34] R. Kallman, H. Kimura, J. Natkins, A. Pavlo, A. Rasin, S. Zdonik, E. P. Jones, S. Madden, M. Stonebraker, Y. Zhang *et al.*, "H-store: a high-performance, distributed main memory transaction processing system," *Proceedings of the VLDB Endowment*, vol. 1, no. 2, pp. 1496–1499, 2008.

[35] Y. Wu, C.-Y. Chan, and K.-L. Tan, "Transaction healing: Scaling optimistic concurrency control on multicores," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 1689–1704.

[36] C. Yao, D. Agrawal, G. Chen, Q. Lin, B. C. Ooi, W.-F. Wong, and M. Zhang, "Exploiting single-threaded model in multi-core in-memory systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 10, pp. 2635–2650, 2016.

[37] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger, "The notions of consistency and predicate locks in a database system," *Communications of the ACM*, vol. 19, no. 11, pp. 624–633, 1976.

[38] H.-T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*

(TODS), vol. 6, no. 2, pp. 213–226, 1981.
[39] R. Sethi, "Useless actions make a difference: Strict serializability of database updates," *Journal of the ACM (JACM)*, vol. 29, no. 2, pp. 394–403, 1982.
[40] C. Gorenflo, L. Golab, and S. Keshav, "Xox fabric: A hybrid approach to blockchain transaction execution," in *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, 2020, pp. 1–9.
[41] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 931–948.
[42] S. Rahnama, S. Gupta, R. Sogani, D. Krishnan, and M. Sadoghi, "Ringbft: Resilient consensus over sharded ring topology," *arXiv preprint arXiv:2107.13047*, 2021.
[43] J. Qi, X. Chen, Y. Jiang, J. Jiang, T. Shen, S. Zhao, S. Wang, G. Zhang, L. Chen, M. H. Au *et al.*, "Bidl: A high-throughput, low-latency permissioned blockchain framework for datacenter networks," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 18–34.
[44] "MultiVAC," https://www.mtv.ac/, 2020.
[45] P. Thakkar and S. Natarajan, "Scaling blockchains using pipelined execution and sparse peers," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 489–502.
[46] S. Gupta, S. Rahnama, E. Linsenmayer, F. Nawab, and M. Sadoghi, "Reliable transactions in serverless-edge architecture," *arXiv preprint arXiv:2201.00982*, 2022.
[47] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: Bft consensus with linearity and responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, 2019, pp. 347–356.
[48] S. Gupta, S. Rahnama, J. Hellings, and M. Sadoghi, "Resilientdb: Global scale resilient blockchain fabric," *arXiv preprint arXiv:2002.00160*, 2020.
[49] E. Buchman, "Tendermint: Byzantine fault tolerance in the age of blockchains," Ph.D. dissertation, 2016.
[50] A. Thomson and D. J. Abadi, "The case for determinism in database systems," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 70–80, 2010.
[51] A. Thomson, T. Diamond, S.-C. Weng, K. Ren, P. Shao, and D. J. Abadi, "Calvin: fast distributed transactions for partitioned database systems," in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, 2012, pp. 1–12.
[52] J. M. Faleiro, D. Abadi, and J. M. Hellerstein, "High performance transactions via early write visibility," *Proc. VLDB Endow.*, vol. 10, no. 5, pp. 613–624, 2017.
[53] Z. Li, P. Romano, and P. Van Roy, "Sparkle: Speculative deterministic concurrency control for partially replicated transactional stores," in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2019, pp. 164–175.
[54] Y. Lu, X. Yu, L. Cao, and S. Madden, "Aria: A fast and practical deterministic OLTP database," *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 2047–2060, 2020.
[55] D. Qin, A. D. Brown, and A. Goel, "Caracal: Contention management with deterministic concurrency control," in *SOSP*. ACM, 2021, pp. 180–194.
[56] S. T. Barnard and H. D. Simon, "Fast multilevel implementation of recursive spectral bisection for partitioning unstructured problems," *Concurrency: Practice and experience*, vol. 6, no. 2, pp. 101–117, 1994.
[57] "Metis," http://glaros.dtc.umn.edu/gkhome/metis/metis/overview, 2020.
[58] "Slither," https://github.com/crytic/slither, 2021.
[59] P. Ruan, G. Chen, T. T. A. Dinh, Q. Lin, B. C. Ooi, and M. Zhang, "Fine-grained, secure and efficient data provenance on blockchain systems," *Proceedings of the VLDB Endowment*, vol. 12, no. 9, pp. 975–988, 2019.
[60] S. Gupta, J. Hellings, and M. Sadoghi, "Rcc: Resilient concurrent consensus for high-throughput secure transaction processing," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 1392–1403.
[61] B. Li, W. Xu, M. Z. Abid, T. Distler, and R. Kapitza, "Sarek: Optimistic parallel ordering in byzantine fault tolerance," in *2016 12th European Dependable Computing Conference (EDCC)*. IEEE, 2016, pp. 77–88.
[62] M. Eischer and T. Distler, "Scalable byzantine fault-tolerant state-machine replication on heterogeneous servers," *Computing*, vol. 101, no. 2, pp. 97–118, 2019.
[63] S. Ponnapalli, A. Shah, A. Tai, S. Banerjee, V. Chidambaram, D. Malkhi, and M. Wei, "Scalable and efficient data authentication for decentralized systems," *arXiv preprint arXiv:1909.11590*, 2019.
[64] "SChain appendix," https://sites.google.com/view/icdeschain/, 2023.
[65] J. Feist, G. Grieco, and A. Groce, "Slither: a static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.
[66] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, "Blockbench: A framework for analyzing private blockchains," in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 1085–1100.
[67] "FabricSharp," https://github.com/ooibc88/FabricSharp, 2020.
[68] S. Mu, L. Nelson, W. Lloyd, and J. Li, "Consolidating concurrency control and consensus for commits under conflicts," in *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*, 2016, pp. 517–532.

APPENDIX

A. Transaction aborting

Here, we discuss how to deal with the situation when a transaction is aborted. In SChain, a transaction can only be aborted for two reasons: (i) deterministic logic; and (ii) inaccurate read/write-set analysis. In the first situation, the transaction will be aborted immediately when the abort logic emerges, e.g., an assertion violation caused by insufficient balance of sender. In later situation, worker thread needs to check whether the keys of states accessed by a transaction during real execution are declared in pre-acquired read/write set. If the validation passes, worker thread applies the generated writes. Otherwise, worker thread aborts the transaction. The abort decision of a transaction would cause some troubles under our concurrent scheme. For instance, if executor E aborts transaction $T_{h,i}$ with write set $\mathcal{W}(T_{h,i}) = \{(I_t, L_t)\}$, other executors in L_t will wait forever for E to forward value of I to them. A naive method is to abort transactions in L_t in cascade. However, this will yield a high transaction abort rate.

We rethink why inaccurate read/write set analysis causes transaction to be aborted. Suppose transaction $T_{h,i}$ writes a state I_t not included in $\mathcal{W}(T_{h,i})$ during real execution and another transaction $T_{h,j}$ $(i < j)$ is going to write I_t. At this time, the lock manager doesn't know the write intention of $T_{h,i}$, thus granting lock on I to $T_{h,j}$ first, which violates the ordering locking scheme. To avoid cascading abort, SChain lets $T_{h,j}$ read the original value of I_t. Observe that before $T_{h,i}$ is aborted, $T_{h,j}$ cannot acquire the lock on I_t according to the ordering locking scheme. If $T_{h,j}$ can read the latest update before $T_{h,i}$, the scheme will be still guaranteed. To this end, we add a *pre-write transaction* PT_t and *pre-executor* PE_t for every state I_t in write-set of transaction $T_{h,i}$ as follows.

$$\mathcal{W}(T_{h,i}) = \{(I_t, L_t, PT, PE), \cdots\}$$

The PT_t is the latest transaction in front of $T_{h,i}$ that writes I_t and PE_t is the executor of PT_t. When $T_{h,i}$ is aborted, for each state I_t, current executor E notifies the pre-write executor PE_t to forward the value of I_t to executors in L_t. Then, transactions in L_t can read the latest value of I_t. This progress can be done recursively if the pre-write transaction is aborted as well. Note that if PT_t doesn't exist, which means I_t is not updated by earlier transaction, current executor E will inform storer to send latest value of I_t till block height $h-1$ to executor in L_t. Besides, if state I_t in $T_{h,i}$'s write-set is also in final write set $FW(B_h)$, E still needs to inform pre-executor PE_t to write the update to storer.

Besides, although some states in read/write set are not accessed by a transaction during real execution, we can still commit it safely. Specifically, the transaction requests all locks on states in read/write-set, performs no operations on states that are not really accessed, and then releases them without side-effect.

Example 2. *Fig. 17 presents an example of transaction aborting at block height h. There are three executors $E_1 \sim E_3$ executing transaction $T_{h,2}$, $T_{h,7}$ and $T_{h,9}$ respectively. The transaction $T_{h,9}$ depends on $T_{h,7}$ at state I_1. For some reasons, executor E_2 aborts $T_{h,7}$, thereby E_3 no longer receives value of I_1 from E_2. Instead, E_3 accepts value of I_t generated by an earlier transaction $T_{h,2}$ executed on E_1. Therefore, E_2 sends the next list $\{(E_3, T_{h,9})\}$ in write-set $\mathcal{W}(T_{h,7})$ to E_1, which is appended to next list of write-set $\mathcal{W}(T_{h,2})$. Specifically, '$\vdash$' and '$\emptyset$' mean that there is no pre-write transaction and executor for $T_{h,2}$ on state I_1. Executor E_1 will send value V_1 of I_1 to E_3 after execution. If E_1 has accomplished $T_{h,2}$, it sends value directly. This requires E_1 to save all intermediate versions of I_1 because it may be overlapped by later transaction on E_1. Executors can collect these garbage when the block B_h is finalized successfully.*

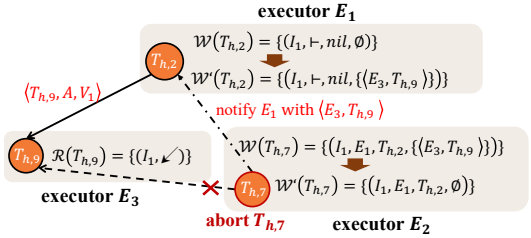


Fig. 17: There is a cross-set dependency $\mathcal{D}(T_{h,9}, T_{h,7}, I_1)$ between transactions $T_{h,9}$ and $T_{h,7}$, and $T_{h,2}$ is the pre-write transaction of $T_{h,7}$ on I_1. Executor E_2 aborts transaction $T_{h,7}$, thus E_1 should send value V_1 of $T_{h,2}$ to $T_{h,9}$ on E_3 instead.

B. Proof of correctness

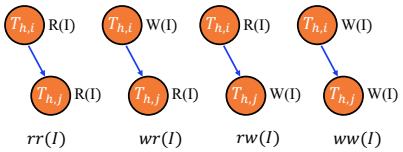


Fig. 18: Conflicts between two transactions.

Theorem 1. *When intra-block concurrency is employed, the effect of writes produced by all executors for B_h is equivalent to the serial order predefined by B_h.*

Proof. The basic idea is to prove that our scheme does not violate the ordering lock scheme as presented in Definition 2. Consider four conflicts, namely $rr(I)$ (read-read), $rw(I)$ (read-write), $wr(I)$ (write-read) and $ww(I)$ (write-write) between two transactions $T_{h,i}, T_{h,j} \in B_h$ as shown in Fig. 18, such that $i < j$. The blue arrow indicates the order of transactions. Since $rr(I)$ conflict is not exclusive, we do not take it into account. Besides, if $T_{h,i}$ and $T_{h,j}$ are assigned to a same executor, the lock manager must grant locks strictly following the ordering lock. Therefore, we just discuss the situation that both transactions are executed by different executors.

If a $rw(I)$ conflict causes the violation of ordering lock, it means $T_{h,j}$ acquires lock on I before $T_{h,i}$ and the write generated by $T_{h,j}$ can is visible to $T_{h,i}$. However, according to the definition of write-set and next list, $T_{h,i}$ cannot appear in the next list of I due to $i < j$. Therefore, the write of $T_{h,j}$ will not be forwarded to $T_{h,i}$ and the violation is impossible.

If a $wr(I)$ conflict causes the violation, it means that $T_{h,j}$ reads the value of I generated by a previous transaction $T_{h,k}$ ($k < i$) or from the storer directly. According to the definition of initial read set $IR(B_h)$, the storer cannot forward value of I to $T_{h,j}$ since $T_{h,j}$'s read appears after $T_{h,i}$'s write. Therefore, $T_{h,j}$ only can read value of I written by $T_{h,k}$. However, the next list of $T_{h,k}$'s write-set cannot contain $T_{h,j}$ due to $k < i$, so the violation is impossible as well. If the $ww(I)$ conflict causes the violation, it means the write of I generated by $T_{h,j}$ is overlapped by $T_{h,i}$. However, according to the definition of final write set $FW(B_h)$, the storer will not accept the write of $T_{h,i}$ since $T_{h,j}$ appears after $T_{h,i}$ in the block B_h.

In summary, the design of SChain will not violate the ordering locking scheme, ensuring that the effect of all changes is equivalent to a deterministic serial order. □